

```

1 //#####
2 // FILE:   FinalProjectStarter_main.c
3 //
4 // TITLE:   Final Project Starter
5 //#####
6
7 // Included Files
8 #include <stdio.h>
9 #include <stdlib.h>
10 #include <stdarg.h>
11 #include <string.h>
12 #include <math.h>
13 #include <limits.h>
14 #include "F28x_Project.h"
15 #include "F2837xD_SWPrioritizedIsrLevels.h"
16 #include "driverlib.h"
17 #include "device.h"
18 #include "F28379dSerial.h"
19 #include "song.h"
20 #include "dsp.h"
21 #include "fpu32/fpu_rfft.h"
22 #include "xy.h"
23 #include "MatrixMath.h"
24 #include "SE423Lib.h"
25 #include "OptiTrack.h"
26
27 #define PI          3.1415926535897932384626433832795
28 #define TWOPI       6.283185307179586476925286766559
29 #define HALFPI      1.5707963267948966192313216916398
30 // The Launchpad's CPU Frequency set to 200 you should not change this value
31 #define LAUNCHPAD_CPU_FREQUENCY 200
32
33 // ----- code for CAN start here -----
34 #include "F28379dCAN.h"
35 // #define TX_MSG_DATA_LENGTH      4
36 // #define TX_MSG_OBJ_ID           0 //transmit
37
38 #define RX_MSG_DATA_LENGTH      8
39 #define RX_MSG_OBJ_ID_1        1 //measurement from sensor 1
40 #define RX_MSG_OBJ_ID_2        2 //measurement from sensor 2
41 #define RX_MSG_OBJ_ID_3        3 //quality from sensor 1
42 #define RX_MSG_OBJ_ID_4        4 //quality from sensor 2
43 // ----- code for CAN end here -----
44
45 // Interrupt Service Routines predefinition
46 __interrupt void cpu_timer0_isr(void);
47 __interrupt void cpu_timer1_isr(void);
48 __interrupt void cpu_timer2_isr(void);
49 __interrupt void SWI1_HighestPriority(void);
50 __interrupt void SWI2_MiddlePriority(void);
51 __interrupt void SWI3_LowestPriority(void);
52 __interrupt void ADCC_ISR(void);
53 __interrupt void SPIB_isr(void);
54 // ----- code for CAN start here -----
55 __interrupt void can_isr(void);
56 // ----- code for CAN end here -----
57

```

```

58 void setF28027EPWM1A(float controleffort);
59 int16_t EPwm1A_F28027 = 1500;
60 void setF28027EPWM2A(float controleffort);
61 int16_t EPwm2A_F28027 = 1500;
62
63 // sjew: centroid to dist fn
64 float centroid_to_dist(float centroid_row);
65
66 // ----- code for CAN start here -----
67 // volatile uint32_t txMsgCount = 0;
68 // extern uint16_t txMsgData[4];
69
70 volatile uint32_t rxMsgCount_1 = 0;
71 volatile uint32_t rxMsgCount_2 = 0;
72 extern uint16_t rxMsgData[8];
73
74 uint32_t dis_raw_1[2];
75 uint32_t dis_raw_2[2];
76 uint32_t dis_1 = 0;
77 uint32_t dis_2 = 0;
78
79 uint32_t quality_raw_1[4];
80 uint32_t quality_raw_2[4];
81 float quality_1 = 0.0;
82 float quality_2 = 0.0;
83
84 uint32_t lightlevel_raw_1[4];
85 uint32_t lightlevel_raw_2[4];
86 float lightlevel_1 = 0.0;
87 float lightlevel_2 = 0.0;
88
89 uint32_t measure_status_1 = 0;
90 uint32_t measure_status_2 = 0;
91
92 volatile uint32_t errorFlag = 0;
93 // ----- code for CAN end here -----
94
95 uint32_t numTimer0calls = 0;
96 uint16_t UARTPrint = 0;
97
98 float printLV1 = 0;
99 float printLV2 = 0;
100
101 float printLinux1 = 0;
102 float printLinux2 = 0;
103
104 uint16_t LADARi = 0;
105 char G_command[] = "G04472503\n"; //command for getting distance -120 to 120 degree
106 uint16_t G_len = 11; //length of command
107 xy ladar_pts[228]; //xy data
108 float LADARrightfront = 0;
109 float LADARfront = 0;
110 float LADARtemp_x = 0;
111 float LADARtemp_y = 0;
112 extern datapts ladar_data[228];
113
114 extern uint16_t newLinuxCommands;

```

```
115 extern float LinuxCommands[CMDNUM_FROM_FLOATS];
116
117 extern uint16_t NewLVData;
118 extern float fromLVvalues[LVNUM_TOFROM_FLOATS];
119 extern LVSendFloats_t DataToLabView;
120 extern char LVsenddata[LVNUM_TOFROM_FLOATS*4+2];
121 extern uint16_t LADARpingpong;
122 extern uint16_t NewCAMDataThreshold1; // Flag new data
123 extern float fromCAMvaluesThreshold1[CAMNUM_FROM_FLOATS];
124 extern uint16_t NewCAMDataThreshold2; // Flag new data
125 extern float fromCAMvaluesThreshold2[CAMNUM_FROM_FLOATS];
126
127 extern uint16_t NewCAMDataAprilTag1; // Flag new data
128 extern float fromCAMvaluesAprilTag1[CAMNUM_FROM_FLOATS];
129
130 float tagid = 0;
131 float tagx = 0;
132 float tagy = 0;
133 float tagz = 0;
134 float tagthetax = 0;
135 float tagthetay = 0;
136 float tagthetaz = 0;
137 int32_t numtag = 0;
138 int32_t AprilTagState = 10;
139 float KpAprilTag = -1.0;
140 float AprilTagSetPoint = 0;
141 int32_t AprilTagCount = 0;
142
143 float MaxAreaThreshold1 = 0;
144 float MaxColThreshold1 = 0;
145 float MaxRowThreshold1 = 0;
146 float NextLargestAreaThreshold1 = 0;
147 float NextLargestColThreshold1 = 0;
148 float NextLargestRowThreshold1 = 0;
149 float NextNextLargestAreaThreshold1 = 0;
150 float NextNextLargestColThreshold1 = 0;
151 float NextNextLargestRowThreshold1 = 0;
152
153 float MaxAreaThreshold2 = 0;
154 float MaxColThreshold2 = 0;
155 float MaxRowThreshold2 = 0;
156 float NextLargestAreaThreshold2 = 0;
157 float NextLargestColThreshold2 = 0;
158 float NextLargestRowThreshold2 = 0;
159 float NextNextLargestAreaThreshold2 = 0;
160 float NextNextLargestColThreshold2 = 0;
161 float NextNextLargestRowThreshold2 = 0;
162
163 uint32_t numThres1 = 0;
164 uint32_t numThres2 = 0;
165
166 pose ROBOTps = {0,0,0}; //robot position
167 pose LADARps = {3.5/12.0,0,1}; // 3.5/12 for front mounting, theta is not used in this
    current code
168 float LADARxoffset = 0;
169 float LADARyoffset = 0;
170
```

```

171 uint32_t timecount = 0;
172 int16_t RobotState = 1;
173 int16_t checkfronttally = 0;
174 int32_t WallFollowtime = 0;
175
176 #define NUMWAYPOINTS 10
177 uint16_t statePos = 0;
178 pose robotdest[NUMWAYPOINTS]; // array of waypoints for the robot
179 uint16_t i = 0; // for loop
180
181 uint16_t right_wall_follow_state = 2; // right follow
182 float Kp_front_wall = -2.0;
183 float front_turn_velocity = 0.2;
184 float left_turn_Stop_threshold = 3.5;
185 float Kp_right_wal = -4.0;
186 float ref_right_wall = 1.1;
187 float foward_velocity = 1.0;
188 float left_turn_Start_threshold = 1.3;
189 float turn_saturation = 2.5;
190
191 float x_pred[3][1] = {{0},{0},{0}}; // predicted state
192
193 // more kalman vars
194 float B[3][2] = {{1,0},{1,0},{0,1}}; // control input model
195 float u[2][1] = {{0},{0}}; // control input in terms of velocity and angular
    velocity
196 float Bu[3][1] = {{0},{0},{0}}; // matrix multiplication of B and u
197 float z[3][1]; // state measurement
198 float eye3[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // 3x3 identity matrix
199 float K[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // optimal Kalman gain
200 #define ProcUncert 0.0001
201 #define CovScalar 10
202 float Q[3][3] = {{ProcUncert,0,ProcUncert/CovScalar},
203                 {0,ProcUncert,ProcUncert/CovScalar},
204                 {ProcUncert/CovScalar,ProcUncert/CovScalar,ProcUncert}}; // process noise
    (covariance of encoders and gyro)
205 #define MeasUncert 1
206 float R[3][3] = {{MeasUncert,0,MeasUncert/CovScalar},
207                 {0,MeasUncert,MeasUncert/CovScalar},
208                 {MeasUncert/CovScalar,MeasUncert/CovScalar,MeasUncert}}; // measurement
    noise (covariance of OptiTrack Motion Capture measurement)
209 float S[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // innovation covariance
210 float S_inv[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // innovation covariance matrix inverse
211 float P_pred[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // predicted covariance (measure of
    uncertainty for current position)
212 float temp_3x3[3][3]; // intermediate storage
213 float temp_3x1[3][1]; // intermediate storage
214 float ytilde[3][1]; // difference between predictions
215
216 // Optitrack Variables
217 int32_t OptiNumUpdatesProcessed = 0;
218 pose OPTITRACKps;
219 extern uint16_t new_optitrack;
220 extern float Optitrackdata[OPTITRACKDATASIZE];
221 int16_t newOPTITRACKpose=0;
222
223 int16_t adcc2result = 0;

```

```
224 int16_t adcc3result = 0;
225 int16_t adcc4result = 0;
226 int16_t adcc5result = 0;
227 float adcC2Volt = 0.0;
228 float adcC3Volt = 0.0;
229 float adcC4Volt = 0.0;
230 float adcC5Volt = 0.0;
231 int32_t numADCCcalls = 0;
232 float LeftWheel = 0;
233 float RightWheel = 0;
234 float LeftWheel_1 = 0;
235 float RightWheel_1 = 0;
236 float LeftVel = 0;
237 float RightVel = 0;
238 float uLeft = 0;
239 float uRight = 0;
240 float HandValue = 0;
241 float vref = 0;
242 float turn = 0;
243 float gyro9250_drift = 0;
244 float gyro9250_angle = 0;
245 float old_gyro9250 = 0;
246 float gyroLPR510_angle = 0;
247 float gyroLPR510_offset = 0;
248 float gyroLPR510_drift = 0;
249 float old_gyroLPR510 = 0;
250 float gyro9250_radians = 0;
251 float gyroLPR510_radians = 0;
252
253 int16_t readdata[25];
254 int16_t IMU_data[9];
255
256 float accelx = 0;
257 float accely = 0;
258 float accelz = 0;
259 float gyrox = 0;
260 float gyroy = 0;
261 float gyroz = 0;
262
263 // Needed global Variables
264 float accelx_offset = 0;
265 float accely_offset = 0;
266 float accelz_offset = 0;
267 float gyrox_offset = 0;
268 float gyroy_offset = 0;
269 float gyroz_offset = 0;
270 int16_t calibration_state = 0;
271 int32_t calibration_count = 0;
272 int16_t doneCal = 0;
273
274 #define MPU9250 1
275 #define DAN28027 2
276 int16_t CurrentChip = MPU9250;
277 int16_t DAN28027Garbage = 0;
278 int16_t dan28027adc1 = 0;
279 int16_t dan28027adc2 = 0;
280 uint16_t MPU9250ignoreCNT = 0; //This is ignoring the first few interrupts if ADCC_ISR and
```

```
    start sending to IMU after these first few interrupts.
281
282 // global vars:
283 float RCangle = 0.0;
284 // sjew global vars:
285 float max_centroid_dist_1 = 0.0;
286 float max_centroid_dist_2 = 0.0;
287 float second_centroid_dist_1 = 0.0;
288 float second_centroid_dist_2 = 0.0;
289 float third_centroid_dist_1 = 0.0;
290 float third_centroid_dist_2 = 0.0;
291 float center_col_const = 80;
292 float Kp_turn_ball = -0.05;
293 float detect_area = 5;
294 int state_cnt = 2000;
295
296 void main(void)
297 {
298     // PLL, WatchDog, enable Peripheral Clocks
299     // This example function is found in the F2837xD_SysCtrl.c file.
300     InitSysCtrl();
301
302     InitGpio();
303
304     InitSE423DefaultGPIO();
305
306     //Scope for Timing
307     GPIO_SetupPinMux(11, GPIO_MUX_CPU1, 0);
308     GPIO_SetupPinOptions(11, GPIO_OUTPUT, GPIO_PUSHPULL);
309     GpioDataRegs.GPACLEAR.bit.GPIO11 = 1;
310
311     GPIO_SetupPinMux(32, GPIO_MUX_CPU1, 0);
312     GPIO_SetupPinOptions(32, GPIO_OUTPUT, GPIO_PUSHPULL);
313     GpioDataRegs.GPBCLEAR.bit.GPIO32 = 1;
314
315     GPIO_SetupPinMux(52, GPIO_MUX_CPU1, 0);
316     GPIO_SetupPinOptions(52, GPIO_OUTPUT, GPIO_PUSHPULL);
317     GpioDataRegs.GPBCLEAR.bit.GPIO52 = 1;
318
319     GPIO_SetupPinMux(60, GPIO_MUX_CPU1, 0);
320     GPIO_SetupPinOptions(60, GPIO_OUTPUT, GPIO_PUSHPULL);
321     GpioDataRegs.GPBCLEAR.bit.GPIO60 = 1;
322
323     GPIO_SetupPinMux(61, GPIO_MUX_CPU1, 0);
324     GPIO_SetupPinOptions(61, GPIO_OUTPUT, GPIO_PUSHPULL);
325     GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;
326
327     GPIO_SetupPinMux(67, GPIO_MUX_CPU1, 0);
328     GPIO_SetupPinOptions(67, GPIO_OUTPUT, GPIO_PUSHPULL);
329     GpioDataRegs.GPCCLEAR.bit.GPIO67 = 1;
330
331 // ----- code for CAN start here -----
332 //GPIO17 - CANRXB
333     GPIO_SetupPinMux(17, GPIO_MUX_CPU1, 2);
334     GPIO_SetupPinOptions(17, GPIO_INPUT, GPIO_ASYNC);
335
336     //GPIO12 - CANTXB
```

```
337     GPIO_SetupPinMux(12, GPIO_MUX_CPU1, 2);
338     GPIO_SetupPinOptions(12, GPIO_OUTPUT, GPIO_PUSH_PULL);
339 // ----- code for CAN end here -----
340
341
342
343 // ----- code for CAN start here -----
344     // Initialize the CAN controller
345     InitCANB();
346
347     // Set up the CAN bus bit rate to 1000 kbps
348     setCANBitRate(200000000, 1000000);
349
350     // Enables Interrupt line 0, Error & Status Change interrupts in CAN_CTL register.
351     CanbRegs.CAN_CTL.bit.IE0= 1;
352     CanbRegs.CAN_CTL.bit.EIE= 1;
353 // ----- code for CAN end here -----
354
355     // Clear all interrupts and initialize PIE vector table:
356     // Disable CPU interrupts
357     DINT;
358
359     // Initialize the PIE control registers to their default state.
360     // The default state is all PIE interrupts disabled and flags
361     // are cleared.
362     // This function is found in the F2837xD_PieCtrl.c file.
363     InitPieCtrl();
364
365     // Disable CPU interrupts and clear all CPU interrupt flags:
366     IER = 0x0000;
367     IFR = 0x0000;
368
369     // Initialize the PIE vector table with pointers to the shell Interrupt
370     // Service Routines (ISR).
371     // This will populate the entire table, even if the interrupt
372     // is not used in this example. This is useful for debug purposes.
373     // The shell ISR routines are found in F2837xD_DefaultIsr.c.
374     // This function is found in F2837xD_PieVect.c.
375     InitPieVectTable();
376
377     // Interrupts that are used in this example are re-mapped to
378     // ISR functions found within this project
379     EALLOW; // This is needed to write to EALLOW protected registers
380     PieVectTable.TIMER0_INT = &cpu_timer0_isr;
381     PieVectTable.TIMER1_INT = &cpu_timer1_isr;
382     PieVectTable.TIMER2_INT = &cpu_timer2_isr;
383     PieVectTable.SCIA_RX_INT = &RXAINT_recv_ready;
384     PieVectTable.SCIB_RX_INT = &RXBINT_recv_ready;
385     PieVectTable.SCIC_RX_INT = &RXCINT_recv_ready;
386     PieVectTable.SCID_RX_INT = &RXDINT_recv_ready;
387     PieVectTable.SCIA_TX_INT = &TXAINT_data_sent;
388     PieVectTable.SCIB_TX_INT = &TXBINT_data_sent;
389     PieVectTable.SCIC_TX_INT = &TXCINT_data_sent;
390     PieVectTable.SCID_TX_INT = &TXDINT_data_sent;
391     PieVectTable.ADCC1_INT = &ADCC_ISR;
392     PieVectTable.SPIB_RX_INT = &SPIB_isr;
393     PieVectTable.EMIF_ERROR_INT = &SWI1_HighestPriority; // Using Interrupt12 interrupts
```



```

    that are not used as SWIs
394     PieVectTable.RAM_CORRECTABLE_ERROR_INT = &SWI2_MiddlePriority;
395     PieVectTable.FLASH_CORRECTABLE_ERROR_INT = &SWI3_LowestPriority;
396     // ----- code for CAN start here -----
397     PieVectTable.CANB0_INT = &can_isr;
398 // ----- code for CAN end here -----
399     EDIS;    // This is needed to disable write to EALLOW protected registers
400
401     // Initialize the CpuTimers Device Peripheral. This function can be
402     // found in F2837xD_CpuTimers.c
403     InitCpuTimers();
404
405     // Configure CPU-Timer 0, 1, and 2 to interrupt every second:
406     // 200MHz CPU Freq,          Period (in uSeconds)
407     ConfigCpuTimer(&CpuTimer0, LAUNCHPAD_CPU_FREQUENCY, 10000); // Currently not used for
any purpose
408     ConfigCpuTimer(&CpuTimer1, LAUNCHPAD_CPU_FREQUENCY, 100000); // !!!!! Important, Used to
command LADAR every 100ms. Do not Change.
409     ConfigCpuTimer(&CpuTimer2, LAUNCHPAD_CPU_FREQUENCY, 40000); // Currently not used for any
purpose
410
411     // Enable CpuTimer Interrupt bit TIE
412     CpuTimer0Regs.TCR.all = 0x4000;
413     CpuTimer1Regs.TCR.all = 0x4000;
414     CpuTimer2Regs.TCR.all = 0x4000;
415
416     DELAY_US(1000000); // Delay 1 second giving Lidar Time to power on after system power on
417
418     init_serialSCIB(&SerialB,19200);
419     init_serialSCIC(&SerialC,19200);
420
421     for (LADARi = 0; LADARi < 228; LADARi++) {
422         ladar_data[LADARi].angle = ((3*LADARi+44)*0.3515625-135)*0.01745329;
//0.017453292519943 is pi/180, multiplication is faster; 0.3515625 is 360/1024
423     }
424
425     init_eQEPs();
426     init_EPWM1and2();
427     init_RCServoPWM_3AB_5AB_6A();
428     init_ADCsAndDACs();
429     setupSpib();
430
431     EALLOW;
432     EPwm4Regs.ETSEL.bit.SOCAEN = 0; // Disable SOC on A group
433     EPwm4Regs.TBCTL.bit.CTRMODE = 3; // freeze counter
434     EPwm4Regs.ETSEL.bit.SOCASEL = 2; // Select Event when counter equal to PRD
435     EPwm4Regs.ETPS.bit.SOCAPRD = 1; // Generate pulse on 1st event
436     EPwm4Regs.TBCTR = 0x0; // Clear counter
437     EPwm4Regs.TBPHS.bit.TBPHS = 0x0000; // Phase is 0
438     EPwm4Regs.TBCTL.bit.PHSEN = 0; // Disable phase loading
439     EPwm4Regs.TBCTL.bit.CLKDIV = 0; // divide by 1 50Mhz Clock
440     EPwm4Regs.TBPRD = 50000; // Set Period to 1ms sample. Input clock is 50MHz.
441     // Notice here that we are not setting CMPA or CMPB because we are not using the PWM
signal
442     EPwm4Regs.ETSEL.bit.SOCAEN = 1; //enable SOCA
443     //EPwm4Regs.TBCTL.bit.CTRMODE = 0; //unfreeze, wait to do this right before enabling
interrupts

```



```

444     EDIS;
445
446     // Enable CPU int1 which is connected to CPU-Timer 0, CPU int13
447     // which is connected to CPU-Timer 1, and CPU int 14, which is connected
448     // to CPU-Timer 2: int 12 is for the SWI.
449     IER |= M_INT1;
450     IER |= M_INT6;
451     IER |= M_INT8; // SCIC SCID
452     IER |= M_INT9; // SCIA
453     IER |= M_INT12;
454     IER |= M_INT13;
455     IER |= M_INT14;
456
457     // Enable TINT0 in the PIE: Group 1 interrupt 7
458     PieCtrlRegs.PIEIER1.bit.INTx7 = 1;
459     PieCtrlRegs.PIEIER1.bit.INTx3 = 1;
460     PieCtrlRegs.PIEIER6.bit.INTx3 = 1;
461     PieCtrlRegs.PIEIER12.bit.INTx9 = 1; //SWI1
462     PieCtrlRegs.PIEIER12.bit.INTx10 = 1; //SWI2
463     PieCtrlRegs.PIEIER12.bit.INTx11 = 1; //SWI3 Lowest priority
464     // ----- code for CAN start here -----
465     // Enable CANB in the PIE: Group 9 interrupt 7
466     PieCtrlRegs.PIEIER9.bit.INTx7 = 1;
467 // ----- code for CAN end here -----
468
469 // ----- code for CAN start here -----
470     // Enable the CAN interrupt signal
471     CanbRegs.CAN_GLB_INT_EN.bit.GLBINT0_EN = 1;
472 // ----- code for CAN end here -----
473
474
475     robotdest[0].x = -4;    robotdest[0].y = 10;
476     robotdest[1].x = -4;    robotdest[1].y = 2;
477     //middle of bottom
478     robotdest[2].x = 0;     robotdest[2].y = 2;
479     //outside the course
480     robotdest[3].x = 0;     robotdest[3].y = -3;
481     //back to middle
482     robotdest[4].x = 0;     robotdest[4].y = 2;
483     robotdest[5].x = 4;     robotdest[5].y = 2;
484     robotdest[6].x = 4;     robotdest[6].y = 10;
485     robotdest[7].x = 0;     robotdest[7].y = 9;
486     robotdest[8].x = 4;     robotdest[8].y = 4;
487     robotdest[9].x = -4;    robotdest[9].y = 4;
488
489     // ROBOTps will be updated by Optitrack during gyro calibration
490     // TODO: specify the starting position of the robot
491     ROBOTps.x = 0;          //the estimate in array form (useful for matrix operations)
492     ROBOTps.y = 0;
493     ROBOTps.theta = 0; // was -PI: need to flip OT ground plane to fix this
494     x_pred[0][0] = ROBOTps.x; //estimate in structure form (useful elsewhere)
495     x_pred[1][0] = ROBOTps.y;
496     x_pred[2][0] = ROBOTps.theta;
497
498     AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //clear interrupt flag
499     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
500     SpibRegs.SPIFFRX.bit.RXFFOVCLR=1; // Clear Overflow flag

```

```
501 SpibRegs.SPIFFRX.bit.RXFFINTCLR=1; // Clear Interrupt flag
502 PieCtrlRegs.PIEACK.all = PIEACK_GROUP6;
503 EPwm4Regs.TBCTL.bit.CTRMODE = 0; //unfreeze, and enter up count mode
504
505 init_serialSCIA(&SerialA,115200);
506 init_serialSCID(&SerialD,2083332);
507
508 // Enable global Interrupts and higher priority real-time debug events
509 EINT; // Enable Global interrupt INTM
510 ERTM; // Enable Global realtime interrupt DBGM
511 // ----- code for CAN start here -----
512 // Measured Distance from 1
513 // Initialize the receive message object 1 used for receiving CAN messages.
514 // Message Object Parameters:
515 //     Message Object ID Number: 1
516 //     Message Identifier: 0x060b0101
517 //     Message Frame: Standard
518 //     Message Type: Receive
519 //     Message ID Mask: 0x0
520 //     Message Object Flags: Receive Interrupt
521 //     Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
522 //     for a Receive mailbox)
523 //
524 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_1, 0x060b0101, CAN_MSG_FRAME_EXT,
525                       CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
526                       RX_MSG_DATA_LENGTH);
527
528 // Measured Distance from 2
529 // Initialize the receive message object 2 used for receiving CAN messages.
530 // Message Object Parameters:
531 //     Message Object ID Number: 2
532 //     Message Identifier: 0x060b0102
533 //     Message Frame: Standard
534 //     Message Type: Receive
535 //     Message ID Mask: 0x0
536 //     Message Object Flags: Receive Interrupt
537 //     Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
538 //     for a Receive mailbox)
539 //
540
541 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_2, 0x060b0102, CAN_MSG_FRAME_EXT,
542                       CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
543                       RX_MSG_DATA_LENGTH);
544
545 // Measurement Quality from 1
546 // Initialize the receive message object 2 used for receiving CAN messages.
547 // Message Object Parameters:
548 //     Message Object ID Number: 3
549 //     Message Identifier: 0x060b0201
550 //     Message Frame: Standard
551 //     Message Type: Receive
552 //     Message ID Mask: 0x0
553 //     Message Object Flags: Receive Interrupt
554 //     Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
555 //     for a Receive mailbox)
556 //
557
```

```

558 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_3, 0x060b0201, CAN_MSG_FRAME_EXT,
559                        CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
560                        RX_MSG_DATA_LENGTH);
561
562 // Measurement Quality from 2
563 // Initialize the receive message object 2 used for receiving CAN messages.
564 // Message Object Parameters:
565 //     Message Object ID Number: 4
566 //     Message Identifier: 0x060b0202
567 //     Message Frame: Standard
568 //     Message Type: Receive
569 //     Message ID Mask: 0x0
570 //     Message Object Flags: Receive Interrupt
571 //     Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
572 //     for a Receive mailbox)
573 //
574
575 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_4, 0x060b0202, CAN_MSG_FRAME_EXT,
576                        CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
577                        RX_MSG_DATA_LENGTH);
578
579 //
580 // Start CAN module operations
581 //
582 CanbRegs.CAN_CTL.bit.Init = 0;
583 CanbRegs.CAN_CTL.bit.CCE = 0;
584
585
586 // ----- code for CAN end here -----
587 char S_command[19] = "S1152000124000\n"; //this change the baud rate to 115200
588 uint16_t S_len = 19;
589 serial_sendSCIC(&SerialC, S_command, S_len);
590
591
592 DELAY_US(1000000); // Delay letting Lidar change its Baud rate
593 init_serialSCIC(&SerialC, 115200);
594
595
596 // LED1 is GPIO22
597 GpioDataRegs.GPACLEAR.bit.GPIO22 = 1;
598 // LED2 is GPIO94
599 GpioDataRegs.GPCCLEAR.bit.GPIO94 = 1;
600 // LED3 is GPIO95
601 GpioDataRegs.GPCCLEAR.bit.GPIO95 = 1;
602 // LED4 is GPIO97
603 GpioDataRegs.GPDCLEAR.bit.GPIO97 = 1;
604 // LED5 is GPIO111
605 GpioDataRegs.GPDCLEAR.bit.GPIO111 = 1;
606
607
608 // IDLE loop. Just sit and loop forever (optional):
609 while(1)
610 {
611     if (UARTPrint == 1 ) {
612         if (readbuttons() == 0) {
613             UART_printfLine(1, "State: %d, cnt: %d", RobotState, state_cnt);
614             UART_printfLine(2, "1: %.2f 2: %.2f", max_centroid_dist_1,

```

```

        second_centroid_dist_2);
615         } else if (readbuttons() == 1) {
616             UART_printfLine(1,"01A:%.0fC:%.0fR:
%.0f",MaxAreaThreshold1,MaxColThreshold1,MaxRowThreshold1);
617             UART_printfLine(2,"P1A:%.0fC:%.0fR:
%.0f",MaxAreaThreshold2,MaxColThreshold2,MaxRowThreshold2);
618             //UART_printfLine(1,"LV1:%.3f LV2:%.3f",printLV1,printLV2);
619             //UART_printfLine(2,"Ln1:%.3f Ln2:%.3f",printLinux1,printLinux2);
620         } else if (readbuttons() == 2) {
621             UART_printfLine(1,"02A:%.0fC:%.0fR:
%.0f",NextLargestAreaThreshold1,NextLargestColThreshold1,NextLargestRowThreshold1);
622             UART_printfLine(2,"P2A:%.0fC:%.0fR:
%.0f",NextLargestAreaThreshold2,NextLargestColThreshold2,NextLargestRowThreshold2);
623             // UART_printfLine(1,"%0.2f %0.2f",adcC2Volt,adcC3Volt);
624             // UART_printfLine(2,"%0.2f %0.2f",adcC4Volt,adcC5Volt);
625         } else if (readbuttons() == 4) {
626             UART_printfLine(1,"03A:%.0fC:%.0fR:
%.0f",NextNextLargestAreaThreshold1,NextNextLargestColThreshold1,NextNextLargestRowThreshold1
);
627             UART_printfLine(2,"P3A:%.0fC:%.0fR:
%.0f",NextNextLargestAreaThreshold2,NextNextLargestColThreshold2,NextNextLargestRowThreshold2
);
628             // UART_printfLine(1,"L:%.3f R:%.3f",LeftVel,RightVel);
629             // UART_printfLine(2,"uL:%.2f uR:%.2f",uLeft,uRight);
630         } else if (readbuttons() == 8) {
631             UART_printfLine(1,"020x%.2f y%.2f",ladar_pts[20].x,ladar_pts[20].y);
632             UART_printfLine(2,"150x%.2f y%.2f",ladar_pts[150].x,ladar_pts[150].y);
633         } else if (readbuttons() == 3) {
634             UART_printfLine(1,"Vrf:%.2f trn:%.2f",vref,turn);
635             UART_printfLine(2,"MPU:%.2f LPR:%.2f",gyro9250_radians,gyroLPR510_radians);
636         } else if (readbuttons() == 5) {
637             UART_printfLine(1,"0x:%.2f:0y:
%.2f:0a%.2f",OPTITRACKps.x,OPTITRACKps.y,OPTITRACKps.theta);
638             UART_printfLine(2,"State:%d : %d",RobotState,statePos);
639         } else if (readbuttons() == 6) {
640             UART_printfLine(1,"D1 %ld D2 %ld",dis_1,dis_2);
641             UART_printfLine(2,"St1 %ld St2 %ld",measure_status_1,measure_status_2);
642         } else if (readbuttons() == 7) {
643             UART_printfLine(1,"%0f,%0.1f,%0.1f,%0.1f",tagid,tagx,tagy,tagz);
644             UART_printfLine(2,"%0.1f,%0.1f,%0.1f",tagthetax,tagthetay,tagthetaz);
645         }
646
647         UARTPrint = 0;
648     }
649 }
650 }
651
652 __interrupt void SPIB_isr(void){
653
654     uint16_t i;
655     GpioDataRegs.GPBSET.bit.GPIO32 = 1;
656
657     GpioDataRegs.GPCSET.bit.GPIO66 = 1;
658     GpioDataRegs.GPASET.bit.GPIO29 = 1; // Pull CS high for DAN28027
659
660     if (CurrentChip == MPU9250) {
661

```

```

662     for (i=0; i<8; i++) {
663         readdata[i] = SpibRegs.SPIRXBUF; // readdata[0] is garbage
664     }
665
666     PostSWI1(); // Manually cause the interrupt for the SWI1
667
668 } else if (CurrentChip == DAN28027) {
669     DAN28027Garbage = SpibRegs.SPIRXBUF;
670     dan28027adc1 = SpibRegs.SPIRXBUF;
671     dan28027adc2 = SpibRegs.SPIRXBUF;
672     CurrentChip = MPU9250;
673     SpibRegs.SPIFFCT.bit.TXDLY = 0;
674 }
675
676 SpibRegs.SPIFFRX.bit.RXFFOVFLR=1; // Clear Overflow flag
677 SpibRegs.SPIFFRX.bit.RXFFINTCLR=1; // Clear Interrupt flag
678 PieCtrlRegs.PIEACK.all = PIEACK_GROUP6;
679 GpioDataRegs.GPBCLEAR.bit.GPIO32 = 1;
680 }
681
682
683
684 //adcd1 pie interrupt
685 __interrupt void ADCC_ISR (void)
686 {
687     GpioDataRegs.GPASET.bit.GPIO11 = 1;
688     adcc2result = AdccResultRegs.ADCRESULT0;
689     adcc3result = AdccResultRegs.ADCRESULT1;
690     adcc4result = AdccResultRegs.ADCRESULT2;
691     adcc5result = AdccResultRegs.ADCRESULT3;
692
693     // Here covert ADCIND0 to volts
694     adcC2Volt = adcc2result*3.0/4095.0;
695     adcC3Volt = adcc3result*3.0/4095.0;
696     adcC4Volt = adcc4result*3.0/4095.0;
697     adcC5Volt = adcc5result*3.0/4095.0;
698
699     if (MPU9250ignoreCNT >= 1) {
700         CurrentChip = MPU9250;
701         SpibRegs.SPIFFCT.bit.TXDLY = 0;
702         SpibRegs.SPIFFRX.bit.RXFFIL = 8;
703
704         GpioDataRegs.GPCCLEAR.bit.GPIO66 = 1;
705
706         SpibRegs.SPITXBUF = ((0x8000)|(0x3A00));
707         SpibRegs.SPITXBUF = 0;
708         SpibRegs.SPITXBUF = 0;
709         SpibRegs.SPITXBUF = 0;
710         SpibRegs.SPITXBUF = 0;
711         SpibRegs.SPITXBUF = 0;
712         SpibRegs.SPITXBUF = 0;
713         SpibRegs.SPITXBUF = 0;
714     } else {
715         MPU9250ignoreCNT++;
716     }
717
718     numADCCcalls++;

```

```
719     AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //clear interrupt flag
720     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
721     GpioDataRegs.GPACLEAR.bit.GPIO11 = 1;
722
723 }
724
725 // cpu_timer0_isr - CPU Timer0 ISR
726 __interrupt void cpu_timer0_isr(void)
727 {
728     CpuTimer0.InterruptCount++;
729
730     numTimer0calls++;
731
732     if ((numTimer0calls%5) == 0) {
733         // Blink LaunchPad Red LED
734         //GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1;
735     }
736
737     // Acknowledge this interrupt to receive more interrupts from group 1
738     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
739 }
740
741 // cpu_timer1_isr - CPU Timer1 ISR
742 __interrupt void cpu_timer1_isr(void)
743 {
744
745     serial_sendSCIC(&SerialC, G_command, G_len);
746
747     CpuTimer1.InterruptCount++;
748 }
749
750 // cpu_timer2_isr CPU Timer2 ISR
751 __interrupt void cpu_timer2_isr(void)
752 {
753     // Blink LaunchPad Blue LED
754     //GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1;
755     PostSWI3();
756     CpuTimer2.InterruptCount++;
757
758     // if ((CpuTimer2.InterruptCount % 10) == 0) {
759     //     UARTPrint = 1;
760     // }
761 }
762
763 void setF28027EPWM1A(float controleffort){
764     if (controleffort < -10) {
765         controleffort = -10;
766     }
767     if (controleffort > 10) {
768         controleffort = 10;
769     }
770     float value = (controleffort+10)*3000.0/20.0;
771     EPwm1A_F28027 = (int16_t)value; // Set global variable that is sent over SPI to F28027
772 }
773 void setF28027EPWM2A(float controleffort){
774     if (controleffort < -10) {
775         controleffort = -10;
```

```

776     }
777     if (controleffort > 10) {
778         controleffort = 10;
779     }
780     float value = (controleffort+10)*3000.0/20.0;
781     EPwm2A_F28027 = (int16_t)value; // Set global variable that is sent over SPI to F28027
782 }
783
784 //
785 // Connected to PIEIER12_9 (use MINT12 and MG12_9 masks):
786 //
787 __interrupt void SWI1_HighestPriority(void)    // EMIF_ERROR
788 {
789     GpioDataRegs.GPASET.bit.GPIO61 = 1;
790     // Set interrupt priority:
791     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
792     IER |= M_INT12;
793     IER &= MINT12; // Set "global" priority
794     PieCtrlRegs.PIEIER12.all &= MG12_9; // Set "group" priority
795     PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
796     __asm(" NOP");
797     EINT;
798
799
800     //#####
801     IMU_data[0] = readdata[1];
802     IMU_data[1] = readdata[2];
803     IMU_data[2] = readdata[3];
804     IMU_data[3] = readdata[5];
805     IMU_data[4] = readdata[6];
806     IMU_data[5] = readdata[7];
807
808     accelx = (((float)(IMU_data[0]))*4.0/32767.0);
809     accely = (((float)(IMU_data[1]))*4.0/32767.0);
810     accelz = (((float)(IMU_data[2]))*4.0/32767.0);
811     gyrox = (((float)(IMU_data[3]))*250.0/32767.0);
812     gyroy = (((float)(IMU_data[4]))*250.0/32767.0);
813     gyroz = (((float)(IMU_data[5]))*250.0/32767.0);
814
815     // g3: calculating distance to centroids, if theres no centroid the distance should be
816     // negative or something right?
817     if(MaxColThreshold1 != 0 || MaxRowThreshold1 != 0) {
818         max_centroid_dist_1 = centroid_to_dist(MaxRowThreshold1); // g3: solve eq here
819     }
820     if(NextLargestColThreshold1 != 0 || NextLargestRowThreshold1 != 0) {
821         second_centroid_dist_1 = centroid_to_dist(NextLargestRowThreshold1); // g3: solve eq
822         here
823     }
824     if(NextNextLargestColThreshold1 != 0 || NextNextLargestRowThreshold1 != 0) {
825         third_centroid_dist_1 = centroid_to_dist(NextNextLargestRowThreshold1); // g3: solve
826         eq here
827     }

```



```
828     if(NextLargestColThreshold2 != 0 || NextLargestRowThreshold2 != 0) {
829         second_centroid_dist_2 = centroid_to_dist(NextLargestRowThreshold2); // g3: solve eq
      here
830     }
831     if(NextNextLargestColThreshold2 != 0 || NextNextLargestRowThreshold2 != 0) {
832         third_centroid_dist_2 = centroid_to_dist(NextNextLargestRowThreshold2); // g3: solve
      eq here
833     }
834
835
836
837     if(calibration_state == 0){
838         calibration_count++;
839         if (calibration_count == 2000) {
840             calibration_state = 1;
841             calibration_count = 0;
842         }
843     } else if(calibration_state == 1){
844         accelx_offset+=accelx;
845         accely_offset+=accely;
846         accelz_offset+=accelz;
847         gyrox_offset+=gyrox;
848         gyroy_offset+=gyroy;
849         gyroz_offset+=gyroz;
850         gyroLPR510_offset+=adcC5Volt;
851         calibration_count++;
852         if (calibration_count == 2000) {
853             calibration_state = 2;
854             accelx_offset/=2000.0;
855             accely_offset/=2000.0;
856             accelz_offset/=2000.0;
857             gyrox_offset/=2000.0;
858             gyroy_offset/=2000.0;
859             gyroz_offset/=2000.0;
860             gyroLPR510_offset/=2000.0;
861             calibration_count = 0;
862         }
863     } else if(calibration_state == 2) {
864
865         gyroLPR510_drift += (((adcC5Volt-gyroLPR510_offset) + old_gyroLPR510)*.0005)/(2000);
866         old_gyroLPR510 = adcC5Volt-gyroLPR510_offset;
867
868         gyro9250_drift += (((gyroz-gyroz_offset) + old_gyro9250)*.0005)/(2000);
869         old_gyro9250 = gyroz-gyroz_offset;
870         calibration_count++;
871
872         if (calibration_count == 2000) {
873             calibration_state = 3;
874             calibration_count = 0;
875             doneCal = 1;
876             newOPTITRACKpose = 0;
877         }
878     } else if(calibration_state == 3){
879         accelx -=(accelx_offset);
880         accely -=(accely_offset);
881         accelz -=(accelz_offset);
882         gyrox -= gyrox_offset;
```

```
883     gyroy -= gyroy_offset;
884     gyroz -= gyroz_offset;
885     LeftWheel = readEncLeft();
886     RightWheel = readEncRight();
887     HandValue = readEncWheel();
888
889     gyro9250_angle = gyro9250_angle + (gyroz + old_gyro9250)*.0005 - gyro9250_drift;
890     old_gyro9250 = gyroz;
891
892     gyroLPR510_angle = gyroLPR510_angle + ((adcC5Volt-gyroLPR510_offset) +
old_gyroLPR510)*.0005 - gyroLPR510_drift;
893     old_gyroLPR510 = adcC5Volt-gyroLPR510_offset;
894
895     gyro9250_radians = (gyro9250_angle * (PI/180.0));
896     gyroLPR510_radians = gyroLPR510_angle * 400 * (PI/180.0);
897
898     LeftVel = (1.235/12.0)*(LeftWheel - LeftWheel_1)*1000;
899     RightVel = (1.235/12.0)*(RightWheel - RightWheel_1)*1000;
900
901     if (newLinuxCommands == 1) {
902         newLinuxCommands = 0;
903         printLinux1 = LinuxCommands[0];
904         printLinux2 = LinuxCommands[1];
905         //?? = LinuxCommands[2];
906         //?? = LinuxCommands[3];
907         //?? = LinuxCommands[4];
908         //?? = LinuxCommands[5];
909         //?? = LinuxCommands[6];
910         //?? = LinuxCommands[7];
911         //?? = LinuxCommands[8];
912         //?? = LinuxCommands[9];
913         //?? = LinuxCommands[10];
914     }
915
916     if (NewCAMDataThreshold1 == 1) {
917         NewCAMDataThreshold1 = 0;
918         MaxAreaThreshold1 = fromCAMvaluesThreshold1[0];
919         MaxColThreshold1 = fromCAMvaluesThreshold1[1];
920         MaxRowThreshold1 = fromCAMvaluesThreshold1[2];
921
922         NextLargestAreaThreshold1 = fromCAMvaluesThreshold1[3];
923         NextLargestColThreshold1 = fromCAMvaluesThreshold1[4];
924         NextLargestRowThreshold1 = fromCAMvaluesThreshold1[5];
925
926         NextNextLargestAreaThreshold1 = fromCAMvaluesThreshold1[6];
927         NextNextLargestColThreshold1 = fromCAMvaluesThreshold1[7];
928         NextNextLargestRowThreshold1 = fromCAMvaluesThreshold1[8];
929         numThres1++;
930         if ((numThres1 % 5) == 0) {
931             // LED4 is GPIO97
932             GpioDataRegs.GPDTOGGLE.bit.GPIO97 = 1;
933         }
934     }
935
936     if (NewCAMDataThreshold2 == 1) {
937         NewCAMDataThreshold2 = 0;
938         MaxAreaThreshold2 = fromCAMvaluesThreshold2[0];
```

```

939         MaxColThreshold2 = fromCAMvaluesThreshold2[1];
940         MaxRowThreshold2 = fromCAMvaluesThreshold2[2];
941
942         NextLargestAreaThreshold2 = fromCAMvaluesThreshold2[3];
943         NextLargestColThreshold2 = fromCAMvaluesThreshold2[4];
944         NextLargestRowThreshold2 = fromCAMvaluesThreshold2[5];
945
946         NextNextLargestAreaThreshold2 = fromCAMvaluesThreshold2[6];
947         NextNextLargestColThreshold2 = fromCAMvaluesThreshold2[7];
948         NextNextLargestRowThreshold2 = fromCAMvaluesThreshold2[8];
949         numThres2++;
950         if ((numThres2 % 5) == 0) {
951             // LED5 is GPIO111
952             GpioDataRegs.GPDTOGGLE.bit.GPIO111 = 1;
953         }
954     }
955
956     if (NewCAMDataAprilTag1 == 1) {
957         NewCAMDataAprilTag1 = 0;
958         tagx = fromCAMvaluesAprilTag1[0];
959         tagy = fromCAMvaluesAprilTag1[1];
960         tagz = fromCAMvaluesAprilTag1[2];
961
962         tagthetax = fromCAMvaluesAprilTag1[3];
963         tagthetay = fromCAMvaluesAprilTag1[4];
964         tagthetaz = fromCAMvaluesAprilTag1[5];
965
966         tagid = fromCAMvaluesAprilTag1[6];
967
968         numtag++;
969         if ((numtag % 5) == 0) {
970             // LED2 is GPIO94
971             GpioDataRegs.GPCTOGGLE.bit.GPIO94 = 1;
972         }
973     }
974
975     if (NewLVData == 1) {
976         NewLVData = 0;
977         printLV1 = fromLVvalues[0];
978         printLV2 = fromLVvalues[1];
979         //?? = fromLVvalues[2];
980         //?? = fromLVvalues[3];
981         //?? = fromLVvalues[4];
982         //?? = fromLVvalues[5];
983         //?? = fromLVvalues[6];
984         //?? = fromLVvalues[7];
985     }
986
987
988     if (new_optitrack == 1) {
989         OPTITRACKps = UpdateOptitrackStates(ROBOTps, &newOPTITRACKpose);
990         new_optitrack = 0;
991     }
992
993     // Step 0: update B, u
994     B[0][0] = cosf(ROBOTps.theta)*0.001;
995     B[1][0] = sinf(ROBOTps.theta)*0.001;

```

```

996     B[2][1] = 0.001;
997     u[0][0] = 0.5*(LeftVel + RightVel);    // linear velocity of robot
998     u[1][0] = gyro*(PI/180.0);    // angular velocity in rad/s (negative for right hand
angle)
999
1000    // Step 1: predict the state and estimate covariance
1001    Matrix3x2_Mult(B, u, Bu);    // Bu = B*u
1002    Matrix3x1_Add(x_pred, Bu, x_pred, 1.0, 1.0); // x_pred = x_pred(old) + Bu
1003    Matrix3x3_Add(P_pred, Q, P_pred, 1.0, 1.0); // P_pred = P_pred(old) + Q
1004    // Step 2: if there is a new measurement, then update the state
1005    if (1 == newOPTITRACKpose) {
1006        OptiNumUpdatesProcessed++;    // checking how often OptiTrack is causing a Kalman
update
1007        if ((OptiNumUpdatesProcessed%10)==0) {
1008            // LED 3
1009            GpioDataRegs.GPCTOGGLE.bit.GPIO95 = 1;
1010        }
1011        newOPTITRACKpose = 0;
1012        z[0][0] = OPTITRACKps.x;    // take in the Optitrack Motion Capture measurement
1013        z[1][0] = OPTITRACKps.y;
1014        // fix for OptiTrack problem at 180 degrees
1015        if (cosf(ROBOTps.theta) < -0.99) {
1016            z[2][0] = ROBOTps.theta;
1017        }
1018        else {
1019            z[2][0] = OPTITRACKps.theta;
1020        }
1021        // Step 2a: calculate the innovation/measurement residual, ytilde
1022        Matrix3x1_Add(z, x_pred, ytilde, 1.0, -1.0);    // ytilde = z-x_pred
1023        // Step 2b: calculate innovation covariance, S
1024        Matrix3x3_Add(P_pred, R, S, 1.0, 1.0);    // S = P_pred + R
1025        // Step 2c: calculate the optimal Kalman gain, K
1026        Matrix3x3_Invert(S, S_inv);
1027        Matrix3x3_Mult(P_pred, S_inv, K);    // K = P_pred*
(S^-1)
1028        // Step 2d: update the state estimate x_pred = x_pred(old) + K*ytilde
1029        Matrix3x1_Mult(K, ytilde, temp_3x1);
1030        Matrix3x1_Add(x_pred, temp_3x1, x_pred, 1.0, 1.0);
1031        // Step 2e: update the covariance estimate P_pred = (I-K)*P_pred(old)
1032        Matrix3x3_Add(eye3, K, temp_3x3, 1.0, -1.0);
1033        Matrix3x3_Mult(temp_3x3, P_pred, P_pred);
1034    }    // end of correction step
1035
1036    // set ROBOTps to the updated and corrected Kalman values.
1037    ROBOTps.x = x_pred[0][0];
1038    ROBOTps.y = x_pred[1][0];
1039    ROBOTps.theta = x_pred[2][0];
1040
1041    // Make sure this function is called every time in this function even if you decide
not to use its vref and turn
1042    // uses xy code to step through an array of positions
1043    if( xy_control(&vref, &turn, 1.0, ROBOTps.x, ROBOTps.y, robotdest[statePos].x,
robotdest[statePos].y, ROBOTps.theta, 0.25, 0.5)) {
1044        statePos = (statePos+1)%NUMWAYPOINTS;
1045    }
1046    // state machine
1047    switch (RobotState) {

```

```

1048     case 1:
1049         // vref and turn are the vref and turn returned from xy_control
1050         state_cnt++; //g3 iterate counter
1051         if (LADARfront < 1.2) {
1052             vref = 0.2;
1053             checkfronttally++;
1054             if (checkfronttally > 310) { // check if LADARfront < 1.2 for 310ms or 3
LADAR samples
1055                 RobotState = 10; // Wall follow
1056                 WallFollowtime = 0;
1057                 right_wall_follow_state = 1;
1058             }
1059         } else {
1060             checkfronttally = 0;
1061         }
1062         // g3: if its been 2 seconds since the last grab and the detected value is
greater than the min area, track the ball
1063         if(MaxAreaThreshold1 >= detect_area && state_cnt >= 2000) {
1064             // g3: whichever ball's max area is larger, track that one
1065             if(MaxAreaThreshold1 >= MaxAreaThreshold2 ) {
1066                 RobotState = 20;
1067                 state_cnt = 0;
1068             } else {
1069                 RobotState = 30;
1070                 state_cnt = 0;
1071             }
1072             // g3: if the code enters this and the not the first one, the second ball is
larger than the first and is a valid target
1073         } else if(MaxAreaThreshold2 >= detect_area && state_cnt >= 2000) {
1074             RobotState = 30;
1075             state_cnt = 0;
1076         }
1077
1078         break;
1079     case 10:
1080         if (right_wall_follow_state == 1) {
1081             //Left Turn
1082             turn = Kp_front_wall*(14.5 - LADARfront);
1083             vref = front_turn_velocity;
1084             if (LADARfront > left_turn_Stop_threshold) {
1085                 right_wall_follow_state = 2;
1086             }
1087         } else if (right_wall_follow_state == 2) {
1088             //Right Wall Follow
1089             turn = Kp_right_wal*(ref_right_wall - LADARrightfront);
1090             vref = foward_velocity;
1091             if (LADARfront < left_turn_Start_threshold) {
1092                 right_wall_follow_state = 1;
1093             }
1094         }
1095         if (turn > turn_saturation) {
1096             turn = turn_saturation;
1097         }
1098         if (turn < -turn_saturation) {
1099             turn = -turn_saturation;
1100         }
1101

```

```

1102         WallFollowtime++;
1103         if ( (WallFollowtime > 5000) && (LADARfront > 1.5) ) {
1104             RobotState = 1;
1105             checkfronttally = 0;
1106         }
1107         break;
1108
1109     case 20:
1110         // put vision code here
1111         // g3: if there is no column detected or the area is too small, dont move and
start the timeout counter
1112         if(MaxColThreshold1 == 0 || MaxAreaThreshold1 < 3) {
1113             vref = 0;
1114             turn = 0;
1115             state_cnt++;
1116         } else {
1117             // turn based on p error, always a constant forward vel (reset the timeout
cnt)
1118             float turn_error = center_col_const - MaxColThreshold1;
1119             turn = Kp_turn_ball * turn_error;
1120             vref = 0.75;
1121             state_cnt = 0;
1122         }
1123         // g3: if the ball is close enough then go to grabbing state
1124         if(MaxRowThreshold1 > 100) {
1125             RobotState = 22;
1126             state_cnt = 0;
1127         }
1128         // if timeout then go back to state 1
1129         if(state_cnt > 5000) {
1130             RobotState = 1;
1131             state_cnt = 0;
1132         }
1133         break;
1134     case 22:
1135         // g3: don't move
1136         state_cnt++;
1137         vref = 0.0;
1138         turn = 0;
1139         // g3: after 1 second, enter state 23
1140         if(state_cnt >= 1000) {
1141             RobotState = 24;
1142             state_cnt = 0;
1143         }
1144         break;
1145     case 24:
1146         // g3: move forward 1s
1147         state_cnt++;
1148         vref = 0.5;
1149         turn = 0;
1150         if (state_cnt >= 1500) {
1151             RobotState = 26;
1152             state_cnt = 0;
1153         }
1154         break;
1155     case 26:
1156         // g3: wait 1s

```

```
1157         state_cnt++;
1158         vref = 0.0;
1159         turn = 0;
1160         if (state_cnt >= 1000) {
1161             RobotState = 1;
1162             state_cnt = 0;
1163         }
1164         break;
1165 // g3: states 30-36 have same functionality as states 20-26, just using threshold 2
1166 case 30:
1167     // put vision code here
1168     if(MaxColThreshold2 == 0 || MaxAreaThreshold2 < 3) {
1169         vref = 0;
1170         turn = 0;
1171         state_cnt++;
1172     } else {
1173         float turn_error = center_col_const - MaxColThreshold2;
1174         turn = Kp_turn_ball * turn_error;
1175         vref = 0.75;
1176     }
1177     if(MaxRowThreshold2 > 100) {
1178         RobotState = 32;
1179         state_cnt = 0;
1180     }
1181     if(state_cnt > 5000) {
1182         RobotState = 1;
1183         state_cnt = 0;
1184     }
1185     break;
1186 case 32:
1187     state_cnt++;
1188     vref = 0.0;
1189     turn = 0;
1190     if(state_cnt >= 1000) {
1191         RobotState = 34;
1192         state_cnt = 0;
1193     }
1194     break;
1195 case 34:
1196     state_cnt++;
1197     vref = 0.5;
1198     turn = 0;
1199     if (state_cnt >= 1500) {
1200         RobotState = 36;
1201         state_cnt = 0;
1202     }
1203     break;
1204 case 36:
1205     state_cnt++;
1206     vref = 0.0;
1207     turn = 0;
1208     if (state_cnt >= 1000) {
1209         RobotState = 1;
1210         state_cnt = 0;
1211     }
1212     break;
1213 default:
```



```

1214         break;
1215     }
1216
1217
1218     //Must be called each time into this SWI1 function
1219     PIcontrol(&uLeft,&uRight,vref,turn,LeftWheel,RightWheel);
1220     // These below lines also must be called each time into this SWI1 function
1221     setEPWM1A(uLeft);
1222     setEPWM2A(-uRight);
1223     setF28027EPWM1A(uLeft);
1224     setF28027EPWM2A(-uRight);
1225     LeftWheel_1 = LeftWheel;
1226     RightWheel_1 = RightWheel;
1227
1228     if((timecount%250) == 0) {
1229         DataToLabView.floatData[0] = ROBOTps.x;
1230         DataToLabView.floatData[1] = ROBOTps.y;
1231         DataToLabView.floatData[2] = ROBOTps.theta;
1232         DataToLabView.floatData[3] = (float)timecount;
1233         DataToLabView.floatData[4] = 0.5*(LeftVel + RightVel);
1234         DataToLabView.floatData[5] = (float)RobotState;
1235         DataToLabView.floatData[6] = (float)statePos;
1236         DataToLabView.floatData[7] = LADARfront;
1237         LVsenddata[0] = '*'; // header for LVdata
1238         LVsenddata[1] = '$';
1239         for (i=0;i<LVNUM_TOFROM_FLOATS*4;i++) {
1240             if (i%2==0) {
1241                 LVsenddata[i+2] = DataToLabView.rawData[i/2] & 0xFF;
1242             } else {
1243                 LVsenddata[i+2] = (DataToLabView.rawData[i/2]>>8) & 0xFF;
1244             }
1245         }
1246         serial_sendSCID(&SerialID, LVsenddata, 4*LVNUM_TOFROM_FLOATS + 2);
1247     }
1248
1249 }
1250 timecount++;
1251 if((timecount%200) == 0)
1252 {
1253     if(doneCal == 0) {
1254         GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1; // Blink Blue LED while calibrating
1255     }
1256     GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1; // Always Block Red LED
1257     UARTPrint = 1; // Tell While loop to print
1258 }
1259
1260 SpibRegs.SPIFFCT.bit.TXDLY = 16;
1261 CurrentChip = DAN28027;
1262 SpibRegs.SPIFFRX.bit.RXFFIL = 3;
1263 GpioDataRegs.GPACLEAR.bit.GPIO29 = 1;
1264 SpibRegs.SPITXBUF = 0x00DA;
1265 SpibRegs.SPITXBUF = EPwm1A_F28027;
1266 SpibRegs.SPITXBUF = EPwm2A_F28027;
1267
1268
1269
1270 //#####

```

```

#####
1270 //
1271 // Restore registers saved:
1272 //
1273 DINT;
1274 PieCtrlRegs.PIEIER12.all = TempPIEIER;
1275
1276 GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;
1277 }
1278
1279 //
1280 // Connected to PIEIER12_10 (use MINT12 and MG12_10 masks):
1281 //
1282 __interrupt void SWI2_MiddlePriority(void) // RAM_CORRECTABLE_ERROR
1283 {
1284 // Set interrupt priority:
1285 volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
1286 IER |= M_INT12;
1287 IER &= MINT12; // Set "global" priority
1288 PieCtrlRegs.PIEIER12.all &= MG12_10; // Set "group" priority
1289 PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
1290 __asm(" NOP");
1291 EINT;
1292
1293
1294 //#####
1295 #####
1296 // Insert SWI ISR Code here.....
1297 // LED1
1298 GpioDataRegs.GPATOGGLE.bit.GPIO22 = 1;
1299 if (LADARpingpong == 1) {
1300 // LADARrightfront is the min of dist 52, 53, 54, 55, 56
1301 LADARrightfront = 19; // 19 is greater than max feet
1302 for (LADARi = 52; LADARi <= 56 ; LADARi++) {
1303 if (ladar_data[LADARi].distance_ping < LADARrightfront) {
1304 LADARrightfront = ladar_data[LADARi].distance_ping;
1305 }
1306 }
1307 // LADARfront is the min of dist 111, 112, 113, 114, 115
1308 LADARfront = 19;
1309 for (LADARi = 111; LADARi <= 115 ; LADARi++) {
1310 if (ladar_data[LADARi].distance_ping < LADARfront) {
1311 LADARfront = ladar_data[LADARi].distance_ping;
1312 }
1313 }
1314 LADARxoffset = ROBOTps.x + (LADARps.x*cosf(ROBOTps.theta)-
1315 LADARps.y*sinf(ROBOTps.theta - PI/2.0));
1316 LADARyoffset = ROBOTps.y + (LADARps.x*sinf(ROBOTps.theta)-
1317 LADARps.y*cosf(ROBOTps.theta - PI/2.0));
1318 for (LADARi = 0; LADARi < 228; LADARi++) {
1319 ladar_pts[LADARi].x = LADARxoffset + ladar_data
1320 [LADARi].distance_ping*cosf(ladar_data[LADARi].angle + ROBOTps.theta);
1321 ladar_pts[LADARi].y = LADARyoffset + ladar_data
1322 [LADARi].distance_ping*sinf(ladar_data[LADARi].angle + ROBOTps.theta);
1323 }
1324 }

```

```

1320     } else if (LADARpingpong == 0) {
1321         // LADARrightfront is the min of dist 52, 53, 54, 55, 56
1322         LADARrightfront = 19; // 19 is greater than max feet
1323         for (LADARi = 52; LADARi <= 56 ; LADARi++) {
1324             if (ladar_data[LADARi].distance_pong < LADARrightfront) {
1325                 LADARrightfront = ladar_data[LADARi].distance_pong;
1326             }
1327         }
1328         // LADARfront is the min of dist 111, 112, 113, 114, 115
1329         LADARfront = 19;
1330         for (LADARi = 111; LADARi <= 115 ; LADARi++) {
1331             if (ladar_data[LADARi].distance_pong < LADARfront) {
1332                 LADARfront = ladar_data[LADARi].distance_pong;
1333             }
1334         }
1335         LADARxoffset = ROBOTps.x + (LADARps.x*cosf(ROBOTps.theta)-
LADARps.y*sinf(ROBOTps.theta - PI/2.0));
1336         LADARYoffset = ROBOTps.y + (LADARps.x*sinf(ROBOTps.theta)-
LADARps.y*cosf(ROBOTps.theta - PI/2.0));
1337         for (LADARi = 0; LADARi < 228; LADARi++) {
1338
1339             ladar_pts[LADARi].x = LADARxoffset + ladar_data
1340 [LADARi].distance_pong*cosf(ladar_data[LADARi].angle + ROBOTps.theta);
1341             ladar_pts[LADARi].y = LADARYoffset + ladar_data
1342 [LADARi].distance_pong*sinf(ladar_data[LADARi].angle + ROBOTps.theta);
1343         }
1344     }
1345
1346
1347
1348
1349     //#####
1350     // Restore registers saved:
1351     //
1352     DINT;
1353     PieCtrlRegs.PIEIER12.all = TempPIEIER;
1354
1355 }
1356
1357 //
1358 // Connected to PIEIER12_11 (use MINT12 and MG12_11 masks):
1359 //
1360 __interrupt void SWI3_LowestPriority(void)    // FLASH_CORRECTABLE_ERROR
1361 {
1362     // Set interrupt priority:
1363     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
1364     IER |= M_INT12;
1365     IER &= MINT12;
1366     PieCtrlRegs.PIEIER12.all &= MG12_11; // Set "global" priority
1367     PieCtrlRegs.PIEACK.all = 0xFFFF;    // Set "group" priority
1368     __asm(" NOP");
1369     EINT;
1370

```

```

1371 //#####
1372 // Insert SWI ISR Code here.....
1373 RCangle = readEncWheel();
1374 if(RCangle < -90)
1375     RCangle = -90;
1376 if(RCangle > 90)
1377     RCangle = 90;
1378 setEPWM3A_RCServo(RCangle); //RCangle is a value from -90 to 90
1379 setEPWM3B_RCServo(RCangle); //RCangle is a value from -90 to 90
1380 setEPWM5A_RCServo(RCangle); //RCangle is a value from -90 to 90
1381 setEPWM5B_RCServo(RCangle); //RCangle is a value from -90 to 90
1382 setEPWM6A_RCServo(RCangle); //RCangle is a value from -90 to 90
1383 //#####
1384 //
1385 // Restore registers saved:
1386 //
1387 DINT;
1388 PieCtrlRegs.PIEIER12.all = TempPIEIER;
1389
1390 }
1391
1392 // ----- code for CAN start here -----
1393 __interrupt void can_isr(void)
1394 {
1395     int i = 0;
1396
1397     uint32_t status;
1398
1399     //
1400     // Read the CAN interrupt status to find the cause of the interrupt
1401     //
1402     status = CANgetInterruptCause(CANB_BASE);
1403
1404     //
1405     // If the cause is a controller status interrupt, then get the status
1406     //
1407     if(status == CAN_INT_INT0ID_STATUS)
1408     {
1409         //
1410         // Read the controller status. This will return a field of status
1411         // error bits that can indicate various errors. Error processing
1412         // is not done in this example for simplicity. Refer to the
1413         // API documentation for details about the error status bits.
1414         // The act of reading this status will clear the interrupt.
1415         //
1416         status = CANgetStatus(CANB_BASE);
1417     }
1418
1419     //
1420     // Check if the cause is the receive message object 2
1421     //
1422     else if(status == RX_MSG_OBJ_ID_1)

```

```
1424     {
1425         //
1426         // Get the received message
1427         //
1428         CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_1, rxMsgData);
1429
1430         for(i = 0; i<2; i++)
1431         {
1432             dis_raw_1[i] = rxMsgData[i];
1433         }
1434
1435         dis_1 = 256*dis_raw_1[1] + dis_raw_1[0];
1436
1437         measure_status_1 = rxMsgData[2];
1438
1439         //
1440         // Getting to this point means that the RX interrupt occurred on
1441         // message object 2, and the message RX is complete. Clear the
1442         // message object interrupt.
1443         //
1444         CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_1);
1445
1446         //
1447         // Increment a counter to keep track of how many messages have been
1448         // received. In a real application this could be used to set flags to
1449         // indicate when a message is received.
1450         //
1451         rxMsgCount_1++;
1452
1453         //
1454         // Since the message was received, clear any error flags.
1455         //
1456         errorFlag = 0;
1457     }
1458
1459     else if(status == RX_MSG_OBJ_ID_2)
1460     {
1461         //
1462         // Get the received message
1463         //
1464         CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_2, rxMsgData);
1465
1466         for(i = 0; i<2; i++)
1467         {
1468             dis_raw_2[i] = rxMsgData[i];
1469         }
1470
1471         dis_2 = 256*dis_raw_2[1] + dis_raw_2[0];
1472
1473         measure_status_2 = rxMsgData[2];
1474
1475         //
1476         // Getting to this point means that the RX interrupt occurred on
1477         // message object 2, and the message RX is complete. Clear the
1478         // message object interrupt.
1479         //
1480         CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_2);
```

```
1481
1482     //
1483     // Increment a counter to keep track of how many messages have been
1484     // received. In a real application this could be used to set flags to
1485     // indicate when a message is received.
1486     //
1487     rxMsgCount_2++;
1488
1489     //
1490     // Since the message was received, clear any error flags.
1491     //
1492     errorFlag = 0;
1493 }
1494
1495 else if(status == RX_MSG_OBJ_ID_3)
1496 {
1497     //
1498     // Get the received message
1499     //
1500     CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_3, rxMsgData);
1501
1502     for(i = 0; i<4; i++)
1503     {
1504         lightlevel_raw_1[i] = rxMsgData[i];
1505         quality_raw_1[i] = rxMsgData[i+4];
1506     }
1507
1508     lightlevel_1 = ((256.0*256.0*256.0)*lightlevel_raw_1[3] +
1509 (256.0*256.0)*lightlevel_raw_1[2] + 256.0*lightlevel_raw_1[1] + lightlevel_raw_1[0])/65535;
1510     quality_1 = ((256.0*256.0*256.0)*quality_raw_1[3] + (256.0*256.0)*quality_raw_1[2] +
1511 256.0*quality_raw_1[1] + quality_raw_1[0])/65535;
1512
1513     //
1514     // Getting to this point means that the RX interrupt occurred on
1515     // message object 2, and the message RX is complete. Clear the
1516     // message object interrupt.
1517     //
1518     CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_3);
1519
1520     //
1521     // Since the message was received, clear any error flags.
1522     //
1523     errorFlag = 0;
1524 }
1525
1526
1527 else if(status == RX_MSG_OBJ_ID_4)
1528 {
1529     //
1530     // Get the received message
1531     //
1532     CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_4, rxMsgData);
1533
1534     for(i = 0; i<4; i++)
1535     {
```

```
1536         lightlevel_raw_2[i] = rxMsgData[i];
1537         quality_raw_2[i] = rxMsgData[i+4];
1538
1539     }
1540
1541     lightlevel_2 = ((256.0*256.0*256.0)*lightlevel_raw_2[3] +
1542 (256.0*256.0)*lightlevel_raw_2[2] + 256.0*lightlevel_raw_2[1] + lightlevel_raw_2[0])/65535;
1543     quality_2 = ((256.0*256.0*256.0)*quality_raw_2[3] + (256.0*256.0)*quality_raw_2[2] +
1544 256.0*quality_raw_2[1] + quality_raw_2[0])/65535;
1545
1546     //
1547     // Getting to this point means that the RX interrupt occurred on
1548     // message object 2, and the message RX is complete. Clear the
1549     // message object interrupt.
1550     //
1551     CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_4);
1552
1553     //
1554     // Since the message was received, clear any error flags.
1555     //
1556     errorFlag = 0;
1557 }
1558
1559 //
1560 // If something unexpected caused the interrupt, this would handle it.
1561 //
1562 else
1563 {
1564     //
1565     // Spurious interrupt handling can go here.
1566     //
1567 }
1568
1569 //
1570 // Clear the global interrupt flag for the CAN interrupt line
1571 //
1572 CANclearGlobalInterruptStatus(CANB_BASE, CAN_GLOBAL_INT_CANINT0);
1573
1574 //
1575 // Acknowledge this interrupt located in group 9
1576 //
1577 InterruptclearACKGroup(INTERRUPT_ACK_GROUP9);
1578 }
1579 // ----- code for CAN end here -----
1580
1581 // g3: calculate the actual distance from robot given centroid row
1582 float centroid_to_dist(float centroid_row) {
1583     float cubic = centroid_row * centroid_row * centroid_row;
1584     float square = centroid_row * centroid_row;
1585     return 0.0008809 * square - 0.1873*centroid_row + 11.22;
1586 }
1587
```